

Business Analytics Architecture, Google Antigravity IDE Initialization, and Core Arithmetic

Bridging Spreadsheet Intuition to Reproducible Python Scripts for Non-Technical Business Students

LECTURE PHASE

60 Minutes

HANDS-ON LAB PHASE

120 Minutes

DELIVERY RHYTHM

1:2 Ratio

PREREQUISITES

Zero Prior Coding

Course Architecture: 4 Modules x 4 Sessions

A structured progression from foundational Python syntax to autonomous predictive business systems.

MODULE 1

Weeks 1-4

Python Core & Pandas

- **Session 1.1:** Antigravity IDE & Core Arithmetic.
- **Session 1.2:** Syntax Completion (Types & Loops).
- **Session 1.3:** Tabular Data (NumPy & Pandas).
- **Session 1.4:** Advanced Filtering & Auditing.

MODULE 2

Weeks 5-8

Web Scraping & Plots

- **Session 2.1:** Scraping with Beautiful Soup.
- **Session 2.2:** Visual Charts (Matplotlib/Seaborn).
- **Session 2.3:** Midterm Problem Formulation.
- **Session 2.4:** Midterm Code Pipeline Defense.

MODULE 3

Weeks 9-12

Machine Learning Code

- **Session 3.1:** Enterprise Artificial Intelligence.
- **Session 3.2:** Feature Preprocessing Pipelines.
- **Session 3.3:** Scikit-Learn Predictive Models.
- **Session 3.4:** Classification Evaluation Metrics.

MODULE 4

Weeks 13-16

AI Decisions & Governance

- **Session 4.1:** End-to-End Decision Architecture.
- **Session 4.2:** Predictive Churn Mitigation.
- **Session 4.3:** Dynamic Pricing Optimization.
- **Session 4.4:** Algorithmic Bias & AI Governance.

THE PROGRAMMING LEARNING CURVE

Students master foundational Python syntax across Weeks 1 and 2, enabling Weeks 3 and 4 to dive straight into large tabular corporate datasets with Pandas.

Core Keywords: Translating Code to Business Metrics

Demystifying programming terms into practical balance-sheet and operational concepts.

CODING TOOL

Infrastructure

Integrated Development Environment (IDE)

A unified software workstation (Google Antigravity IDE) that combines file management, text editing, terminal execution, and AI pair programming in one workspace.

- **Interactive Execution:** Allows running scripts and seeing live calculation outputs immediately in the terminal.
- **Auditability:** Code, documentation, and raw datasets live together in a single structured directory.

COMPUTATIONAL CONCEPT

Variables

Variable Assignment & Data Types

A variable is a labeled container in computer memory storing a piece of business information (numbers or text) for automated calculation.

- **Integer & Float:** Whole numbers (units sold) and decimal numbers (revenue in USD millions).
- **Business Meaning:** Naming values descriptive names like ``net_sales`` replaces grid coordinates like ``C13``.

COMMERCIAL METRIC

Volume

Gross Merchandise Volume (GMV)

The total monetary volume of merchandise transacted across a commercial platform before platform commission deductions or customer discounts.

- **Take Rate:** The percentage of GMV retained by the platform as recognized Net Revenue.
- **Algorithmic Tracking:** Python scripts calculate GMV across millions of transactions in seconds.

PERFORMANCE METRIC

Efficiency

Operating Margin Percentage (Operating Margin Pct)

The proportion of enterprise revenue remaining after operating expenses: Operating Income divided by Net Sales multiplied by 100.

- **Fixed Cost Leverage:** Demonstrates how operating profits surge when revenues scale faster than operating expenses.
- **Return on Investment (ROI):** Measures net operational profit generated per capital dollar spent.

THE NON-TECHNICAL REASSURANCE

In this course, programming syntax is never taught as abstract software engineering; every single script directly calculates, verifies, or models a tangible business performance metric.

The Mental Model: Spreadsheets vs. Python Code

Translating fragile cell coordinates into robust, reproducible, and automated data pipelines.

SPREADSHEET PARADIGM

Excel Mechanics

Spatial Grid Thinking

- **Grid References:** Formulas reference abstract coordinates like `=C13-E13` and `=(E13/C13)*100`, hiding business logic.
- **Manual Repetition:** Applying logic requires dragging formulas down rows, creating silent formula drift and copy errors.
- **Data Fragility:** Raw financial data and calculated formulas share the same space; accidental keystrokes overwrite raw numbers.
- **Scale Ceiling:** Spreadsheets freeze and crash when enterprise transactional datasets exceed 500,000 rows.

PYTHON PIPELINE PARADIGM

Code Mechanics

Sequential Recipe Thinking

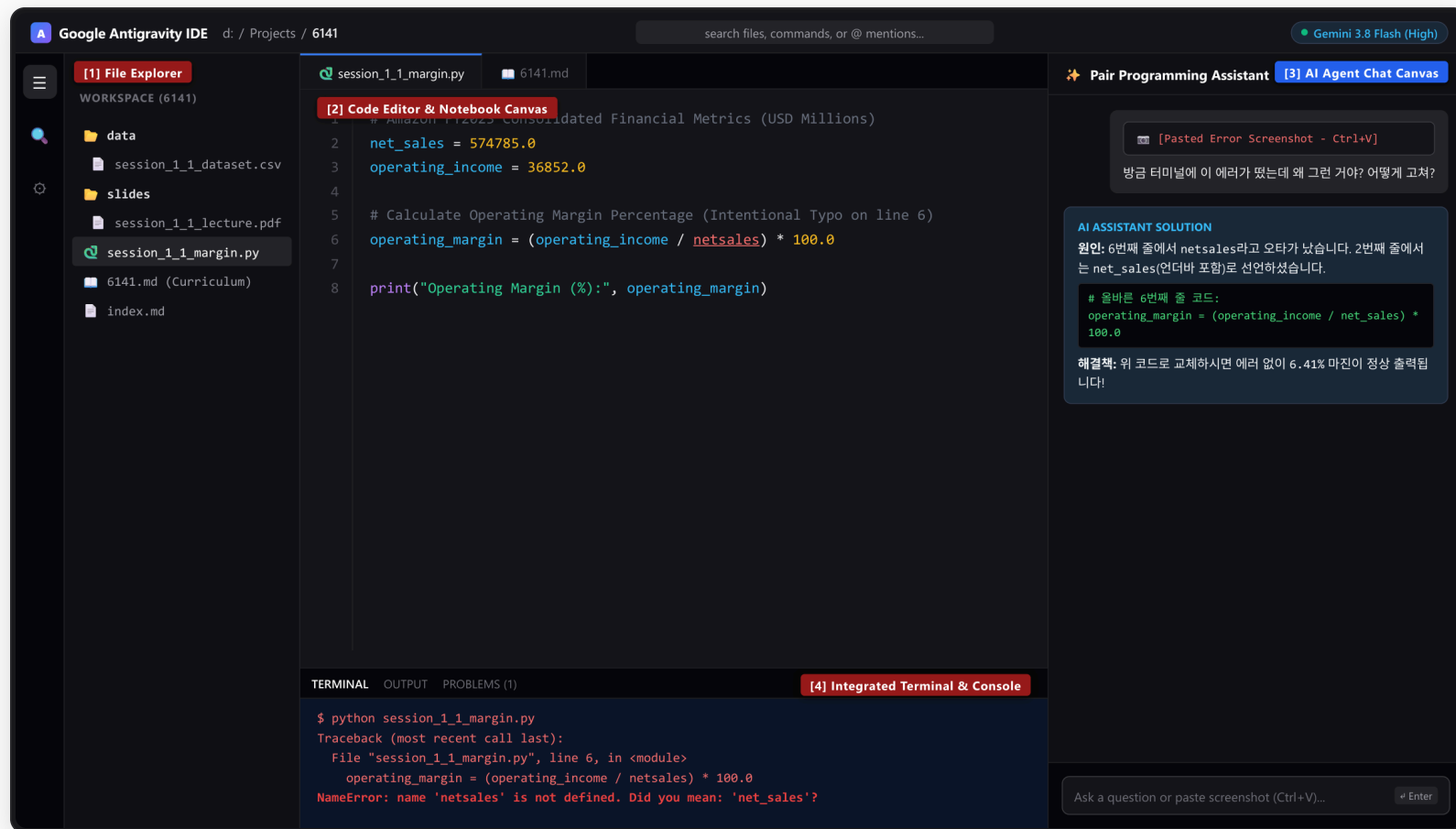
- **Self-Documenting Logic:** Code reads like English: `operating_expense = net_sales - operating_income`.
- **Data Immutability:** Raw corporate CSV files are read-only; scripts never overwrite primary source data.
- **One-Click Execution:** A written script executes across 10 rows or 10,000,000 rows with identical speed and zero fatigue.
- **Complete Audit Trail:** Every transformation, filter, and aggregation step is fully logged and verifiable for auditors.

CORE PEDAGOGICAL INSIGHT

Think of a Python script not as complex software engineering, but as an automated, error-proof recipe that performs your spreadsheet calculations at lightning speed.

Google Antigravity IDE: Architecture & AI Diagnostics

Four integrated workspace panels and the zero-friction screen capture error resolution protocol.



WORKSPACE LAYOUT

4 Core Panels

Unified Coding Workstation

- [1] File Explorer:** Navigate datasets ('data/'), python scripts, and slides.
- [2] Code Editor:** Write arithmetic formulas and multi-line business code.
- [3] AI Agent Chat:** Intelligent pair-programming assistant powered by Gemini.
- [4] Terminal:** Execution console displaying calculation output and error tracebacks.

ZERO-FRICTION TROUBLESHOOTING

Win + Shift + S

AI Error Diagnosis Protocol

- 1. Spot the Traceback:** Python shows the exact line number in Terminal '[4]'.
- 2. Capture Snippet:** Press 'Win + Shift + S' (macOS: 'Cmd + Shift + 4').
- 3. Paste into AI Chat:** Press 'Ctrl + V' in Panel '[3]' and ask: "Why did this happen?"
- 4. Copy Verified Solution:** AI explains root cause and provides clean code.

THE NON-TECHNICAL ANALYST PRINCIPLE

Errors are not failures; they are GPS coordinates. Never struggle alone with code when your AI pair programmer is one screenshot away.

First Python Calculations: Auditing Amazon Financials

Executable multi-line Python syntax paired line-by-line with managerial accounting anatomy.

```
amazon_margin_audit.py • Antigravity Editor

1  # Amazon FY2023 Consolidated Financials (USD M)
2  net_sales = 574785.0
3  operating_income = 36852.0
4
5  # Calculate Operating Expenses and Margin Pct
6  operating_expense = net_sales - operating_income
7  operating_margin_pct = (operating_income / net_sales) * 100.0
8
9  # Output executive formatted metrics
  print("Operating Margin (%)ate", round(operating_margin_pct, 2))

Terminal Output • Return Code: 0 (Success)      >>> Operating Margin (%): 6.41
```

Lines 1 - 3

Variable Assignment (=)

Stores raw financial dollars from Note 15 of Amazon's SEC Form 10-K into memory variables. Replaces grid coordinates like `C13` with descriptive words.

Lines 5 - 7

Financial Formula Equivalence

Identical to Excel `=C13-E13` and `=(E13/C13)\*100`. Python computes exact floating-point decimals with zero manual drag-and-drop cell corruption.

Lines 9 - 10

Formatting Function (round)

`round(..., 2)` formats the raw number to standard 2-decimal financial precision (`6.41%`), creating clean boardroom-ready metric summaries.

THE EXCEL EQUIVALENCE RULE

Cell formula `=C13-E13` becomes `net\_sales - operating\_income`. The arithmetic is identical, but the Python statement explains its own business purpose to anyone reading the code.

Reading Python Errors Without Fear: Diagnostic Anatomy

Common beginner mistakes paired line-by-line with root-cause diagnostic explanations.

```
debugging_practice.py • Antigravity Editor

1  # 1. SyntaxError: Unclosed parenthesis
2  print("Amazon Operating Margin: " + str(margin)
3
4  # 2. NameError: Inconsistent variable spelling
5  net_sales = 574785.0
6  operating_margin = 36852.0 / netsales
7
8  # 3. TypeError: Adding text string to number
9  total_cost = 500 + "100"
10

Terminal Diagnosis • Exit 1: Exception      Traceback: Line 2, Line 6, Line 10
```

Lines 1 - 3: SyntaxError

Unclosed Parenthesis Rule

Two opening parentheses `print(` and `str(` require two closing parentheses `))`. Python halts immediately when syntax rules are broken.

Lines 5 - 7: NameError

Variable Misspelling & Case Sensitivity

Python cannot find `netsales` because it was defined as `net\_sales`. Press `Tab` in Antigravity to auto-complete variable names safely.

Lines 9 - 10: TypeError

Data Type Incompatibility

Python refuses to perform mathematical addition between a number (`500`) and text (`"100"`). The fix: convert text with `float("100")`.

THE PROFESSIONAL DEBUGGING MINDSET

An error traceback is not a test failure; it is Python politely explaining the exact line number and cause of a discrepancy so you can fix it in seconds.

Lab Transition: 120-Minute Hands-On Coding Protocol

Applying the 1:2 delivery rhythm: shifting from lecture instruction to guided code execution and AI debugging.

EXERCISE 1

Guided Practice

Writing Your First Financial Script

Step-by-step guided protocol executing your first Python statements in Google Antigravity IDE.

- **Step 1:** Launch Google Antigravity IDE and open ``session_1_1_margin.py``.
- **Step 2:** Define financial variables: ``net_sales = 574785.0``, ``operating_income = 36852.0``.
- **Step 3:** Write arithmetic formulas: ``operating_expense`` and ``operating_margin_pct``.
- **Step 4:** Run script in Terminal ``[4]`` and verify formatted output: ``6.41%``.

EXERCISE 2+

Coding Challenge

Multi-Unit Margins & AI Debugging

Hands-on practice expanding financial scripts and mastering error resolution with AI.

- **Task 1 (Segment Calculation):** Define AWS variables (``aws_sales = 90757.0``, ``aws_income = 24631.0``) and compute cloud operating margin (``27.14%``).
- **Task 2 (Intentional Error):** Introduce an intentional typo (such as ``netsales``), observe the terminal ``NameError``, capture with ``Win + Shift + S``, and paste into AI Chat.
- **Task 3 (Executive Summary Output):** Print a formatted 2-line comparison of consolidated vs. cloud margins with zero syntax errors.

THE PEDAGOGICAL CONTRACT

Mastery in business programming comes not from memorizing complex syntax, but from building computational confidence: writing clean arithmetic statements, reading error tracebacks calmly, and collaborating with your AI assistant.